Week 3 - CloudFormation change control (change sets, drift detection, stack policies, termination protection, DeletionPolicy / UpdateReplacePolicy)

DOP-C02 Domain 2 (Configuration Management and IaC). These features answer one exam question in many forms: **how do we change stacks safely and stop accidents?** Each feature protects against a **different** accident, and the exam tests whether you choose the right one.

Part 1 - Which feature protects against what

Risk	Feature	What it does
Don't know what an update will change or replace	**Change sets**	**Preview** an update before running it
Someone changed resources outside CloudFormation	**Drift detection**	**Finds** differences between template and real resources
An update might replace or delete the production database	**Stack policy**	**Blocks update actions** on chosen resources
Someone might delete the whole stack	**Termination protection**	**Blocks** `DeleteStack`
A resource is removed from the template, or the stack is deleted, and data is lost	**DeletionPolicy**	**Retain / Snapshot** on removal
An update replaces a resource and the old one (with data) is deleted	**UpdateReplacePolicy**	**Retain / Snapshot** the **old** resource on replacement
Resource-level protection	Service settings	RDS `DeletionProtection`, S3 versioning / Object Lock, DynamoDB deletion protection
Who may do any of this	**IAM** (and SCPs)	Limits who can update, execute change sets, delete stacks or change policies

Part 2 - Change sets

A **change set** is a **preview** of what CloudFormation will do with a new template or new parameters. **Nothing changes until you execute it.**

```
# 1. Create the preview
aws cloudformation create-change-set --stack-name orders-prod \
  --change-set-name release-42 --template-body file://template.yaml \
  --parameters ParameterKey=InstanceType,ParameterValue=m5.large \
  --capabilities CAPABILITY_IAM

# 2. Read it
aws cloudformation describe-change-set --stack-name orders-prod --change-set-name release-42

# 3a. Apply it            or     3b. Throw it away
aws cloudformation execute-change-set --stack-name orders-prod --change-set-name release-42
aws cloudformation delete-change-set  --stack-name orders-prod --change-set-name release-42
```

What a change set shows for each resource

Field	Values	Meaning
Action	`Add`, `Modify`, `Remove`, `Import`, `Dynamic`	What happens. **Dynamic** = can't be known until execution
Replacement	`True`, `False`, `Conditional`	**True = new physical resource, old one deleted** (data loss risk). Conditional = depends on values known only at execution
Scope	`Properties`, `Metadata`, `Tags`, `CreationPolicy`, `UpdatePolicy`, `DeletionPolicy`	Which part changed
Details	Property name, `RequiresRecreation` (`Always` / `Conditionally` / `Never`), causing entity	**Why** it changes

Rules to remember

- A change set shows **intent**. It **doesn't guarantee success**: permission errors, quotas or name conflicts appear at execution.
- **Executing** a change set makes the **other change sets** for that stack obsolete.
- If the stack changes after a change set was created, that change set **can't be executed**. Create a new one.
- **Types:** `UPDATE` (default), `CREATE` (preview a **new** stack; status `REVIEW_IN_PROGRESS`), `IMPORT` (bring existing resources in, Part 7).
- **Nested stacks:** `--include-nested-stacks` shows changes inside child stacks.
- `aws cloudformation deploy` always uses a change set; `--no-execute-changeset` creates the preview only.
- **Separation of duties:** developers get `cloudformation:CreateChangeSet` / `DescribeChangeSet`; only approvers or the pipeline get `cloudformation:ExecuteChangeSet`.
- Change sets don't change **termination protection, stack policy** or stack tags; those have their own commands.

Change sets in CodePipeline

```
Source -> Build -> Deploy stage:
   1. CloudFormation action  ActionMode: CHANGE_SET_REPLACE   (create / replace change set)
   2. Manual approval        (reviewer reads the change set: any Replacement = True?)
   3. CloudFormation action  ActionMode: CHANGE_SET_EXECUTE   (apply it)
```

Action mode	What it does
`CREATE_UPDATE`	Create or update the stack **directly** (no review)
`CHANGE_SET_REPLACE`	Create a change set, or replace one with the same name

Action mode	What it does
`CHANGE_SET_EXECUTE`	Execute that change set
`DELETE_ONLY`	Delete the stack
`REPLACE_ON_FAILURE`	Delete and recreate the stack if it is in a failed state

- The action also takes `TemplatePath`, `TemplateConfiguration` (JSON with parameters, tags and a stack policy), `ParameterOverrides`, `RoleArn` (the **CloudFormation service role**), `Capabilities` and `OutputFileName`.

Part 3 - Drift detection

Drift detection compares the **real configuration** of resources with what the **template expects**, finding changes made **outside CloudFormation** (for example a security group edited in the console).

```
aws cloudformation detect-stack-drift --stack-name orders-prod          # returns a detection ID
aws cloudformation describe-stack-drift-detection-status --stack-drift-detection-id <id>
aws cloudformation describe-stack-resource-drifts --stack-name orders-prod \
  --stack-resource-drift-status-filters MODIFIED DELETED
aws cloudformation detect-stack-resource-drift --stack-name orders-prod --logical-resource-id WebSg
```

Level	Statuses
Stack	`DRIFTED`, `IN_SYNC`, `NOT_CHECKED`, `UNKNOWN`
Resource	`IN_SYNC`, `MODIFIED` (expected vs actual values), `DELETED` (removed outside CloudFormation), `NOT_CHECKED` (type not supported)

Important limits

- **Runs only when you ask**; not continuous by default.
- **Not every resource type is supported** (`NOT_CHECKED`).
- Checks properties **set in the template**; properties left at defaults aren't compared.
- **Only reports, doesn't fix.** To fix: **revert** the manual change, or **update the template** to match reality and deploy, or for deleted resources recreate / remove / **import** again.
- A stack update **doesn't automatically correct drift** on untouched properties, and updates can **fail** because of drift (for example a resource deleted outside CloudFormation).
- Doesn't see changed **secret values** behind dynamic references.
- **StackSets** have their own drift detection across stack instances. For **nested stacks**, detect on the root, then check the children.

Automating drift detection

Method	How
AWS Config managed rule `cloudformation-stack-drift-detection-check`	Marks drifted stacks **NON_COMPLIANT**; add **automatic remediation** (SSM Automation) or notifications. **Common exam answer for continuous drift monitoring**
EventBridge schedule + Lambda	Call `DetectStackDrift` on a schedule and alert through SNS
Prevent drift	IAM / SCP policies that stop direct changes; read-only consoles; changes **only through pipelines**

Part 4 - Stack policies

A **stack policy** is a JSON document attached to a stack that says **which resources may be updated, and how**, during **stack updates**.

```
{
  "Statement": [
    { "Effect": "Allow", "Action": "Update:*", "Principal": "*", "Resource": "*" },
    {
      "Effect": "Deny",
      "Action": ["Update:Replace", "Update:Delete"],
      "Principal": "*",
      "Resource": "LogicalResourceId/ProductionDatabase"
    },
    {
      "Effect": "Deny",
      "Action": "Update:*",
      "Principal": "*",
      "Resource": "*",
      "Condition": { "StringEquals": { "ResourceType": ["AWS::RDS::DBInstance", "AWS::EC2::VPC"] } }
    }
  ]
}
```

Element	Values
Action	`Update:Modify` (in place), `Update:Replace` (replacement), `Update:Delete` (removed from the template), `Update:*`
Principal	Must be `"*"`. Control users with **IAM**
Resource	`LogicalResourceId/<name>` (wildcards allowed) or `*`
Condition	`ResourceType` matching, for example all `AWS::RDS::DBInstance`

Rules to remember

- **Once a stack policy is set, every resource is denied by default.** Explicitly **Allow** what can change, usually `Allow Update:*` on `*` first, then Deny statements.
- **Explicit Deny wins** over Allow.
- A stack policy **can't be removed**, only **replaced** (use an allow-all policy to "remove" it).
- **Temporary override:** `--stack-policy-during-update-body` (or `-url`) applies **only for that update**.
- Protects against **stack updates only**: **not** stack deletion (termination protection), **not** changes outside CloudFormation (IAM to prevent, drift to detect), **not** resources when the whole stack is deleted (DeletionPolicy).
- **Who can change it** is IAM: `cloudformation:SetStackPolicy`.
- A blocked action makes the **update fail and roll back**; the event names the resource.
- Set it at creation (`--stack-policy-body`), later (`set-stack-policy`), or in a CodePipeline **TemplateConfiguration** file.

Part 5 - Termination protection

- When **enabled**, `DeleteStack` **fails** for everyone until someone turns it off.
- **Off by default.** `aws cloudformation update-termination-protection --enable-termination-protection --stack-name orders-prod`.
- **Nested stacks** inherit it from the **root stack**; set it on the root.
- It **doesn't** stop **updates**, including updates that **delete or replace resources** (stack policies and DeletionPolicy do that).
- Stronger: **IAM / SCP deny** on `cloudformation:DeleteStack` and `cloudformation:UpdateTerminationProtection` for production stacks.

Part 6 - DeletionPolicy and UpdateReplacePolicy

DeletionPolicy - when the resource is removed

Applies when **the stack is deleted**, or **the resource is removed from the template** in an update.

Value	What happens	Supported on
`Delete`	Resource deleted. **Default for most resources**	All
`Retain`	Resource **kept**, **no longer managed** (orphaned: you pay and manage it)	All
`Snapshot`	**Final snapshot**, then deleted	`AWS::EC2::Volume`, `AWS::RDS::DBInstance`, `AWS::RDS::DBCluster`, `AWS::Redshift::Cluster`, `AWS::ElastiCache::CacheCluster` / `ReplicationGroup`, `AWS::Neptune::DBCluster`, `AWS::DocDB::DBCluster`
`RetainExceptOnCreate`	**Retain**, except resources created by a stack **creation that failed and rolled back** are **deleted**	All. Avoids orphaned leftovers

- **Default exceptions:** `AWS::RDS::DBCluster`, and `AWS::RDS::DBInstance` without `DBClusterIdentifier`, default to `Snapshot`.
- **S3 buckets:** CloudFormation **can't delete a non-empty bucket**: the stack goes to `DELETE_FAILED`. Retain it, empty it first, or use a custom resource that empties it.

UpdateReplacePolicy - when an update replaces the resource

Value	Old resource
`Delete` (default)	Deleted
`Retain`	Kept (orphaned)
`Snapshot`	Snapshot, then deleted (snapshot-capable types only)

Use both on stateful resources

```
OrdersDb:
  Type: AWS::RDS::DBInstance
  DeletionPolicy: Snapshot          # stack deleted or resource removed -> final snapshot
  UpdateReplacePolicy: Snapshot     # update replaces the DB -> snapshot of the old one
  Properties:
    DeletionProtection: true        # RDS itself refuses deletion (another layer)

LogsBucket:
  Type: AWS::S3::Bucket
  DeletionPolicy: Retain
  UpdateReplacePolicy: Retain
```

- **DeletionPolicy** = **removed**; **UpdateReplacePolicy** = **replaced**. Setting only DeletionPolicy **doesn't protect** data during replacement.
- **cfn-lint warns** when stateful resources lack these policies.
- Both are **resource attributes** next to `Type` and `Properties`, not inside `Properties`.
- **Service-level layers:** RDS / Aurora `DeletionProtection`, DynamoDB `DeletionProtectionEnabled`, EC2 `DisableApiTermination`, S3 **versioning / Object Lock**, ELB deletion protection.

Part 7 - Retain + import (moving and fixing resources)

Resource import (an `IMPORT` change set) brings **existing** resources under a stack:

1. Add the resource to the template **with a** `DeletionPolicy` (required for imports).
2. Create a change set of type **IMPORT** with each resource's identifier (for example `BucketName`).
3. Review and execute.

- **Move a resource between stacks:** `DeletionPolicy: Retain` in the old stack, remove it from that template, then **import** into the new stack. Newer: **stack refactoring** moves resources or renames logical IDs in one operation.
- **Adopt** manually created resources.
- **Fix a resource deleted outside CloudFormation:** recreate and import, or remove it from the template.

Part 8 - Service roles and IAM controls

- **CloudFormation service role** (`--role-arn`): CloudFormation uses this role for resource operations. Users need only CloudFormation permissions plus `iam:PassRole`, not admin on every service.
- **IAM condition keys:** `cloudformation:ChangeSetName`, `cloudformation:ResourceTypes` (limit creatable types), `cloudformation:RoleArn` (force a service role), `cloudformation:TemplateUrl` (approved templates), `cloudformation:StackPolicyUrl`.
- **SCPs** can deny `DeleteStack` / `UpdateTerminationProtection` on production stacks (for example by tag), or deny direct changes to resources tagged as CloudFormation-managed.

Part 9 - Exam traps

Trap	Truth
Preview whether an update will replace the database	**Change set**: look for **Replacement: True**
A change set shows success, so the update will succeed	**Intent only**; errors can still happen
Developers propose changes, only leads apply	IAM **CreateChangeSet** vs **ExecuteChangeSet**, or **CHANGE_SET_REPLACE, manual approval, CHANGE_SET_EXECUTE**
Drift detection fixes drift	**Only reports.** Revert, or update the template
Continuous drift monitoring with alerts	**Config rule** `cloudformation-stack-drift-detection-check` (or scheduled Lambda)
All resources show NOT_CHECKED	**Type not supported**
Stack policy prevents deleting the stack	**No. Termination protection** (+ IAM)
Stack policy stops console changes to the DB	**No.** CloudFormation updates only. IAM prevents, drift detects
One Deny statement added, now no updates work	**Default deny**: add `Allow Update:*` **on** `*`
Remove a stack policy	Can't. **Replace** with allow-all
Update a protected resource once	`--stack-policy-during-update-body`
Limit stack policy to users with Principal	Principal must be `*`; use **IAM**
Termination protection blocks replacement	**No.** Only **DeleteStack**
Termination protection on a nested stack	Set it on the **root stack**
DeletionPolicy Retain protects the DB on replacement	**No.** Needs **UpdateReplacePolicy**
Snapshot on S3 bucket or DynamoDB table	**Not supported. Retain** (+ backups, PITR, versioning)
Stack deletion fails on a bucket	Bucket **not empty**
Failed creation leaves orphaned retained resources	`RetainExceptOnCreate`
Import an existing bucket	**IMPORT change set** + **DeletionPolicy**
Retained resources are still managed	**No**, orphaned. Import again to manage
Deployers need admin only to run CloudFormation	**Service role** + `iam:PassRole`

Part 10 - Hands-on lab (about 60 to 90 minutes, very low cost)

Lab 1 - Change sets and replacement

Create `stack.yaml`:

```
Parameters:
  QueueSuffix: { Type: String, Default: v1 }
Resources:
  DataBucket:
    Type: AWS::S3::Bucket
    DeletionPolicy: Retain
    UpdateReplacePolicy: Retain
  WorkQueue:
    Type: AWS::SQS::Queue
    Properties:
      QueueName: !Sub "${AWS::StackName}-work-${QueueSuffix}"
      VisibilityTimeout: 30
  AppSg:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: lab sg
      SecurityGroupIngress:
        - { IpProtocol: tcp, FromPort: 443, ToPort: 443, CidrIp: 10.0.0.0/8 }
Outputs:
  Bucket: { Value: !Ref DataBucket }
```

1. `aws cloudformation deploy --template-file stack.yaml --stack-name lab-protect`.
2. Change `VisibilityTimeout` to 60 and `deploy --no-execute-changeset`: **Modify, Replacement: False**.
3. Delete that change set. Set `QueueSuffix=v2`: **Replacement: True** (QueueName changed).
4. Execute and watch events: new queue created, old one deleted.

Lab 2 - Drift detection

1. In the EC2 console, add an inbound port 22 rule from `0.0.0.0/0` to the lab security group.
2. **Stack actions, Detect drift**: `AppSg` is **MODIFIED**; view expected vs actual.
3. Remove the rule by hand and detect again: **IN_SYNC**.
4. Optional: enable the Config rule `cloudformation-stack-drift-detection-check` (needs a role).

Lab 3 - Stack policy

1. Apply a policy that **denies** `Update:Replace` **on** `WorkQueue` (with `Allow Update:*` first) using `set-stack-policy`.
2. Try `QueueSuffix=v3` (replacement): **fails and rolls back**, event mentions the stack policy.
3. Change `VisibilityTimeout` (Modify): **succeeds**.
4. Run the replacement with `--stack-policy-during-update-body` allowing all: succeeds; the original policy still applies afterwards.
5. Experiment: a policy with **only** the Deny statement blocks **every** update (default deny).

Lab 4 - Termination protection and DeletionPolicy

1. Enable termination protection; `delete-stack` **fails**.
2. Disable it, put an object in the bucket, delete the stack: the stack goes, **the bucket remains** (Retain).
3. Optional: with `DeletionPolicy: Delete` and an object in the bucket, deletion ends in `DELETE_FAILED`.

Lab 5 - Import the retained bucket

1. Create `import.yaml` with only the bucket (`BucketName: <retained name>`, `DeletionPolicy: Retain`).
2. Create an **IMPORT** change set with `--resources-to-import` (ResourceType `AWS::S3::Bucket`, LogicalResourceId, ResourceIdentifier `BucketName`) and execute it.

Clean up

- Empty and delete the retained bucket, delete the stacks (disable termination protection first), and remove the Config rule if created.

Part 11 - Check yourself

Questions

1. How do you find out, before an update, whether an RDS instance will be replaced?
2. A change set looked fine but execution failed. Possible? Why?
3. How do you let developers propose changes while only leads apply them? Two ways.
4. Someone added a rule to a security group in the console. How do you detect and fix it?
5. The company wants continuous detection of drifted stacks with notifications. What do you use?
6. A stack policy with one statement denying `Update:Replace` on the database was added, and now all updates fail. Why?
7. Does a stack policy stop someone deleting the stack, or changing the database in the RDS console?
8. A protected resource must be replaced once during maintenance. How?
9. What does termination protection block and not block?
10. Difference between DeletionPolicy and UpdateReplacePolicy, with an example of each?
11. Which DeletionPolicy value avoids orphaned resources when stack creation fails?
12. Can you use `DeletionPolicy: Snapshot` on an S3 bucket or DynamoDB table?
13. Stack deletion fails with `DELETE_FAILED` on an S3 bucket. Why, and options?
14. How do you bring a manually created bucket under CloudFormation management?
15. How can deployers run CloudFormation without admin permissions on every service?

Answers

1. Create a **change set** and check for **Replacement: True** (Details: RequiresRecreation).
2. **Yes.** It shows intended changes only; permissions, quotas, name conflicts or invalid values can fail at execution.
3. (a) **IAM:** developers `CreateChangeSet` / `DescribeChangeSet`, leads `ExecuteChangeSet`. (b) **CodePipeline:** `CHANGE_SET_REPLACE`, **manual approval**, `CHANGE_SET_EXECUTE`.
4. **Detect drift** (SG **MODIFIED** with expected vs actual). **Revert** the change or **update the template** and deploy.
5. **Config rule** `cloudformation-stack-drift-detection-check` with SNS or remediation (or a scheduled Lambda).
6. Stack policies **deny everything by default**. Add `Allow Update:*` **on** `*`.
7. **No to both.** Use **termination protection / IAM** for deletes; **IAM** to prevent and drift detection to detect console changes.
8. `--stack-policy-during-update-body` with a temporary allowing policy.
9. Blocks `DeleteStack`. Not updates, replacements, deletion through updates, or outside changes.
10. **DeletionPolicy:** resource **removed** (stack deleted or resource removed from template). **UpdateReplacePolicy:** update **replaces** it, the policy handles the **old** one. Changing an RDS property that requires replacement uses UpdateReplacePolicy; deleting the stack uses DeletionPolicy.
11. `RetainExceptOnCreate`.
12. **No.** Only snapshot-capable types (EBS, RDS, Aurora, Redshift, ElastiCache, Neptune, DocumentDB). Use **Retain** plus versioning, backups or PITR.
13. The bucket **isn't empty**. Empty it, `DeletionPolicy: Retain`, or a custom resource that empties it.
14. Add it with a **DeletionPolicy**, then an **IMPORT change set** with its BucketName.
15. A **CloudFormation service role** (`--role-arn`); deployers need CloudFormation permissions plus `iam:PassRole`.

Quick memory summary

- **Change sets** = preview (Action, **Replacement True / False / Conditional**), intent only, IAM create vs execute, CodePipeline **CHANGE_SET_REPLACE, approval, CHANGE_SET_EXECUTE**, types UPDATE / CREATE / IMPORT.
- **Drift detection** = on demand, reports only (MODIFIED / DELETED / NOT_CHECKED); continuous via **Config** `cloudformation-stack-drift-detection-check`.
- **Stack policy** = updates only, **default deny once set**, Principal `*`, can't remove (replace), temporary override during update.
- **Termination protection** = blocks DeleteStack only; root stack; protect with IAM.
- **DeletionPolicy** (removed: Delete / Retain / Snapshot / RetainExceptOnCreate) vs **UpdateReplacePolicy** (replaced: Delete / Retain / Snapshot). Use both on stateful resources. Non-empty buckets fail deletion.
- **Import** needs a DeletionPolicy. **Service role** + PassRole for least privilege.